

Día D — qué ejecuta cada uno

Ramas y fases ya están repartidas. Cada quien corre **un solo comando de agente**: `/speckit-implement` con su fase. Lo único que se escribe a mano es **la Historia que dicte el profesor**.

Quién	Su rama	Su fase	Archivos que le pertenecen — nadie más los toca
Sergio	<code>main</code>	reparte	<code>spec.md</code> , <code>tasks.md</code>
Mateo	<code>feature/hu-ui</code>	fase 6	<code>app/**/page.jsx</code> , <code>components/**</code> , <code>hooks/**</code>
Johan	<code>feature/hu-api</code>	fase 7	<code>app/api/**/route.js</code> y sus <code>.test.js</code>
Tomás	<code>feature/hu-datos</code>	fase 8	<code>test/**</code> , <code>lib/**</code>
Santiago	<code>main</code>	merge y deploy	<code>scripts/**</code>

Paso 0 · los cinco

20 minutos antes de clase

```
cd ~/Documents/web/speckit-ai-generator
git checkout main
git pull origin main
npm install
npm test # 0 failed
npx -y react-doctor@latest . --score # 100
```

La última línea baja el paquete: si no la corres ahora, en vivo se tarda un minuto largo. Hoy da **100**; el piso del CI es **95** (`.react-doctor-baseline`). Por debajo de 95 no despliega.

Santiago, además — deja el panel de Vercel abierto y la URL cargada en el celular:

```
npm run smoke https://ia-generator-openspec.vercel.app
```

Este smoke es la puerta de entrada, no un trámite. Si `/capture` contesta **404**, lo desplegado es viejo: no tiene la línea base y la demo arranca rota. Se arregla desplegando `main` otra vez, **no en clase**. Verifica también en Vercel que la *Production Branch* del proyecto sea `main`.

Y cada quien deja abierto `specs/001-ai-media-generator/tasks.md` en la fase que le toca: es lo que se señala cuando pregunten de dónde salió el código.

Paso 1 · Sergio mete la Historia y reparte las fases

≈ 2 min · rama `main`

```
git checkout main
git pull origin main
```

1.1 — la Historia entra al spec. Pega esto con la historia (o las dos) literales del profesor:

```
/speckit-specify Agrega al final de la seccion User Scenarios de
specs/001-ai-media-generator/spec.md las User Stories que siguen, numeradas
desde User Story 5.
HU 5: "historia literal que dictó el profesor"
# si dictó una segunda, agrega esta línea; si no, bórrala
HU 6: "segunda historia literal"
Cada una con sus Acceptance Scenarios numerados en Given/When/Then, y agrega
los Functional Requirements que hagan falta.
NO crees una feature nueva. NO corras create-new-feature.sh. NO cambies de rama.
NO escribas ni modifiques .specify/feature.json.
NO crees la carpeta checklists/ ni ningun archivo dentro de ella.
NO toques plan.md ni tasks.md. Solo edita ese spec.md que ya existe.
```

Verifica esto antes de seguir. `/speckit-specify` por dentro dice que *siempre* crea una carpeta nueva y reescribe `.specify/feature.json`. Si lo hace, los tres `/speckit-implement` apuntan a un `tasks.md` vacío y no construyen nada.

```
cat .specify/feature.json # tiene que decir specs/001-ai-media-generator
git status --short # solo spec.md modificado, ninguna carpeta specs/002-...
ls specs/001-ai-media-generator/checklists # tiene que decir: No such file
```

Si salió una carpeta nueva: `rm -rf specs/002-*` y `git checkout .specify/feature.json`. Si salió `checklists/`: `rm -rf specs/001-ai-media-generator/checklists`. Con una sola casilla sin marcar ahí adentro, los tres `/speckit-implement` se **frenan** a preguntar «¿sigue igual? (yes/no)» y los tres quedan esperando en medio de la demo.

1.2 — el reparto. Este prompt se pega tal cual, sin cambiar nada:

Agrega al final de specs/001-ai-media-generator/tasks.md cuatro fases nuevas para las User Stories que acabamos de agregar. No regeneres el archivo ni toques las fases 1 a 5. Numera las tareas desde T040.

```
## Fase 6 – pantallas    Owns: app/**/page.jsx, components/**, hooks/**
## Fase 7 – API          Owns: app/api/**/route.js y sus .test.js
## Fase 8 – datos        Owns: test/**, lib/**
## Fase 9 – merge        Owns: scripts/**
```

Antes de las tareas, escribe el contrato HTTP entre la Fase 6 y la Fase 7: ruta, metodo, forma exacta del cuerpo de respuesta y código para el anónimo. Ese contrato no se negocia después.

Reglas: ningún archivo puede aparecer en dos fases. Cada tarea dice que Acceptance Scenario cubre, con la etiqueta [US5-2], y es test-first: primero la prueba, nombrada it("US5-2: ..."), después el código. Cada tarea manda hacer commit de la prueba en rojo antes de escribir el código, con el mensaje test: US5-2 ..., para que el historial demuestre el test-first. Ninguna fase puede importar un símbolo nuevo de un archivo que le pertenece a otra fase – si lo necesita, lo define en su propio archivo. Ordena las tareas dentro de cada fase para que un archivo se cree antes de que otra tarea lo importe. Si una HU no da trabajo para alguna fase, deja esa fase con una sola tarea que lo diga.

```
git add specs/001-ai-media-generator/
git commit -m "HU en vivo: User Stories nuevas y sus fases 6, 7, 8 y 9"
git push origin main
```

Después dice en voz alta: «fase 6 Mateo, fase 7 Johan, fase 8 Tomás — arranquen».

Paso 2 · Mateo, Johan y Tomás

≈ 6 min · cada uno en su máquina

Copia solo tu columna, de arriba abajo. El agente hace el resto:

MATEO

```
git checkout main
git pull origin main
git checkout -b feature/hu-ui

/speckit-implement fase 6
solo esa fase. no toques
.gitignore, eslint.config.mjs
ni ningún archivo de config.

npm test
npx -y react-doctor@latest . --score
git add -A
git commit -m "feat: HU en vivo pantallas"
git push -u origin feature/hu-ui
```

JOHAN

```
git checkout main
git pull origin main
git checkout -b feature/hu-api

/speckit-implement fase 7
solo esa fase. no toques
.gitignore, eslint.config.mjs
ni ningún archivo de config.

npm test
npx -y react-doctor@latest . --score
git add -A
git commit -m "feat: HU en vivo API"
git push -u origin feature/hu-api
```

TOMÁS

```
git checkout main
git pull origin main
git checkout -b feature/hu-datos

/speckit-implement fase 8
solo esa fase. no toques
.gitignore, eslint.config.mjs
ni ningún archivo de config.

npm test
npx -y react-doctor@latest . --score
git add -A
git commit -m "feat: HU en vivo datos"
git push -u origin feature/hu-datos
```

Es `/speckit-implement`, no `/implement`, y siempre con la fase — sin argumento el agente construye todo el backlog. **Rojo en npm test**, o un score por debajo de 95, es no-push: dile al agente qué falló, que lo arregle y vuelva a correr los dos. Cuando tu push termine, anúncialo en voz alta.

Lo que se rompió en el ensayo, y lo que hay que decirle al agente si pasa.

- **Pantalla nueva que lee datos al montar** — las pruebas viejas de esa pantalla sustituyen `fetch` por orden, y la lectura nueva se come la primera respuesta encolada. Cinco pruebas que nada tienen que ver se ponen rojas. Se arregla en ese mismo archivo: que el doble responda **por método**, no por turno.
- **Score que baja del piso** — `react-doctor` castiga un `fetch` dentro de un `useEffect` en un `page.jsx`, y un `<div onClick>` haciendo de modal. Sácale el `fetch` a un hook en `hooks/` y usa `<dialog>`. También castiga `.filter().map()` encadenados — un `flatMap`. El baseline está en 95, así que hay margen para una advertencia, no para tres — y en el ensayo salieron justo tres.
- **Nombres accesibles que chocan** — dos imágenes con `alt` parecido, o un botón nuevo cuyo nombre accesible sale de su `alt`, rompen `getByRole` de pruebas viejas. Ponle `aria-label` explícito al botón nuevo.
- **Helper de pruebas que no exporta lo nuevo** — si el archivo de la ruta tiene un `load()` que devuelve `{ POST, ...mocks }`, agregar un `GET` sin agregarlo ahí falla con `GET is not a function` y parece un error de la ruta.

- El agente “verificando la configuración del proyecto” — `/speckit-implement` trae un paso que revisa `.gitignore`, `eslint.config.mjs` y compañía, y les agrega lo que crea que falta. Si los tres lo hacen, los tres editan los mismos archivos compartidos y el merge del Paso 3 explota justo donde el reparto por *Owens* lo había evitado. Por eso el comando lleva la línea de «no toques ningún archivo de config».
- Importar algo que otro está escribiendo ahora — nadie importa un símbolo nuevo de `test/fixtures.js` ni de un archivo de otra fase. Datos de ejemplo, en tu propio archivo.

Paso 3 · Santiago junta las tres ramas y despliega

≈ 2 min · rama `main`

Solo cuando los tres hayan dicho que subieron. En este orden — datos, API, pantallas:

```
git checkout main
git pull origin main
git fetch origin
git merge --no-ff origin/feature/hu-datos
git merge --no-ff origin/feature/hu-api
git merge --no-ff origin/feature/hu-ui
```

Uno por línea, esperando a que termine cada uno. En el ensayo los tres entraron sin un solo conflicto: ningún archivo aparece en dos listas de *Owens*. Si un merge se enreda: `git merge --abort`, y esa rama salió de su fase.

Con los tres adentro, los cuatro chequeos — el score también:

```
npm run lint
npm test
npx -y react-doctor@latest . --score
npm run build
git push origin main
```

Ese push despliega en Vercel solo. Los tres `feature/*` también disparan un preview cada uno y en el plan Hobby se construyen de a uno: **el único que importa es el de `main`**, no esperes a los otros tres. Mientras construye, proyecta el grafo de ramas:

```
git log --graph --oneline -12
```

Paso 4 · todos validan sobre el despliegue

≈ 1 min

1. Santiago espera el check verde del deploy en Vercel (1–2 min) y corre:

```
npm run smoke https://ia-generator-openspec.vercel.app
```

2. Todos abren `https://ia-generator-openspec.vercel.app` en el celular, con datos móviles, y recorren la Historia nueva de punta a punta.
3. Si preguntan de dónde salió el código, el dueño abre `tasks.md` en su fase, señala la etiqueta `[US5-2]` y corre esa sola prueba:

```
npm test -- -t "US5-2" # unas pocas passed, el resto skipped
```

- y `git log --oneline` en su rama, donde el commit `test:` va antes del código.
4. Si preguntan qué criterio de aceptación demuestra esa prueba, se lee el `Given/When/Then` numerado en `spec.md`, en la User Story que el profesor acaba de dictar. Ese es el camino completo: **necesidad** → **spec** → **tasks** → **prueba** → **código** → **commit** → **desplegado**.

Si dice que `specs/001-ai-media-generator/` es de solo lectura, la Historia entra como una **feature nueva**, `specs/002-...`. La app funciona igual — el código no sabe dónde vive el spec. Lo que cambia es el Paso 1 de Sergio: pasa de dos prompts pegados a **tres comandos de agente**, porque `setup-tasks.sh` aborta si no existe `plan.md`. Cuenta ≈ 4 min en vez de 2. Los Pasos 0, 3 y 4 no cambian.

B.1 — Sergio, los tres comandos, uno detrás de otro:

```
/speckit-specify historia literal del profesor
```

Es una feature nueva encima del producto que ya existe en `specs/001-ai-media-generator/spec.md`. NO edites esa carpeta, es de solo lectura: lee la para no contradecirla y escribe todo en la nueva. NO crees la carpeta `checklists/`.

```
/speckit-plan
```

Reutiliza la arquitectura de `specs/001-ai-media-generator/plan.md`, no inventes otra. Fija el contrato HTTP y el reparto por propiedad de archivos en tres bloques disjuntos: pantallas, API, y pruebas/datos. NO edites `specs/001-ai-media-generator/`.

```
/speckit-tasks
```

Agrupar las tareas por los tres bloques de propiedad que `plan.md` ya define, no por User Story. Nombra los encabezados "## Bloque A -", "## Bloque B -", "## Bloque C -", nunca "Fase". Cada tarea lleva su etiqueta [US1-N] y es test-first. NO edites `specs/001-ai-media-generator/`.

```
git add specs/ .specify/feature.json
git commit -m "HU en vivo: feature 002"
git push origin main
```

B.2 — verifica antes de repartir:

```
cat .specify/feature.json # specs/002-...
git status --short        # specs/001 sin tocar
ls specs/002-*/           # spec.md plan.md tasks.md
```

La trampa que borra el proyecto. `specs/001-.../tasks.md` también tiene una «Fase 1», y esa Fase 1 arranca con `npx create-next-app`. Si alguien corre `/speckit-implement fase 1` sin haber traído el `.specify/feature.json` nuevo, su agente lee el archivo equivocado y **vuelve a andamiar el proyecto encima del que ya existe**.

Por eso los encabezados de 002 se llaman **bloques**, no fases: «bloque A» no coincide con nada en 001, así que el agente se queda quieto y lo dice.

B.3 — el Paso 2 igual, cambiando solo el argumento. Cada quien hace `git pull` antes, o su agente lee el spec viejo:

Mateo	<code>/speckit-implement bloque A</code>	pantallas
Johan	<code>/speckit-implement bloque B</code>	API
Tomás	<code>/speckit-implement bloque C</code>	pruebas y datos

Las cinco trampas del ensayo del Paso 2 aplican igual: son del código, no del spec.

Si el tiempo aprieta y `/speckit-plan` se demora: un solo prompt que escriba `plan.md` y `tasks.md` de una vez también sirve. Se pierde el comando en pantalla, no la trazabilidad.